

C2_02 Advanced SQL

Astra Parity RC4 Learner Book

Twelve PostgreSQL-centered lessons for grain-safe joins, readable query architecture, windows/time analysis, validation, performance reasoning and maintainable analytical SQL.

Route: Learner Book -> matching P-ASQL task -> Mini-Lab 1 -> Mini-Lab 2 -> Quick Revision -> non-gating Module Test -> protected review -> Gate A. Gate B/C remain fresh reassessments only when explicitly assigned.

Module map

ID	Skills
ASQL1	Advanced joins and grain control
ASQL2	Subqueries and derived tables
ASQL3	CTEs and readable query architecture
ASQL4	Window functions fundamentals
ASQL5	Ranking, partitions and running calculations
ASQL6	LAG, LEAD and time-based comparisons
ASQL7	Conditional aggregation and advanced KPI queries
ASQL8	Date and time analysis
ASQL9	Data-quality investigation with SQL
ASQL10	Query debugging and validation
ASQL11	Performance concepts: indexes, scans and EXPLAIN
ASQL12	Maintainable production-style analytical SQL

Dialect: PostgreSQL behavior/documentation is authoritative. Preserved Alex The Analyst segments are visual companions, not a replacement for PostgreSQL syntax/reference checks.

ASQL1 - Advanced joins and grain control

1. Why it matters

The most dangerous SQL join errors often run successfully and produce believable totals. If a one-to-many or many-to-many relationship was not expected, revenue or counts can multiply silently.

2. What you are learning

State row grain and key/cardinality on both sides before joining. Predict output grain, select the join kind deliberately, surface unmatched rows and prove whether any fan-out is expected.

3. Prerequisite / what you already know

Course 1 SELECT/WHERE/GROUP BY and basic INNER/LEFT JOIN. C2-00 grain language should be familiar.

4. Picture it in your head

Row-multiplication ledger: Before a join, write the row grain on the left and right. Predict how many matches each key can create. If a one-row order meets three item rows, the ledger says three output rows unless you first aggregate to order grain.

5. New words before we use them

Cardinality: How many matching rows may exist on each side of a relationship. Fan-out: Unexpected row multiplication after a join. Orphan key: A foreign/business key with no matching parent/master row.

6. Watch / visual source

RECOMMENDED VIDEO: Alex The Analyst - Learn SQL Beginner to Advanced in Under 4 Hours. 51:10-1:08:03.

<https://www.youtube.com/watch?v=OT1RErkfLNQ>

CURRENT REFERENCE: PostgreSQL 18 - Joins Between Tables. Join concepts.

<https://www.postgresql.org/docs/current/tutorial-join.html>

Return to the lesson and reproduce the SQL/control from your own practice database. Save course-task evidence, not a screenshot of the source.

7. Follow with me once

- Declare orders = one row per order_id. Declare order_items = one row per order_id + line_id.
- Do not join order-level order_value to item rows and then SUM(order_value), because each order value would repeat once per item.
- First aggregate line items to the order grain: WITH line_totals AS (SELECT order_id, SUM(quantity*unit_price) AS line_value FROM order_items GROUP BY order_id).
- Join orders to line_totals on order_id. Predict one output row per order.
- Validate with COUNT(*), COUNT(DISTINCT order_id), unmatched order_ids, and an independent total. If rows > distinct orders, investigate
- instead of adding DISTINCT.

8. What you should see / be able to say

One row per intended order with no unexplained multiplication and a saved orphan/fan-out check.

9. Why it worked

The many-side table was reduced to the required output grain before the join, so the join contract matched the intended result. Professional upgrade:

Production analytical SQL usually documents expected cardinality and validates it. A reviewer should be able to see why row count changed and whether that change was intended.

10. Common beginner mistakes

1. Joining before writing the two row-grain sentences. | 2. Using DISTINCT to hide fan-out. | 3. Using INNER JOIN when unmatched source rows must remain visible. | 4. Validating only the final total without row/key checks.

11. If you get stuck - recovery ladder

Run each table alone. Record rows and distinct business keys. Join only keys first. Compare row count before/after. List duplicates and orphans. Repair the first stage where predicted grain breaks. Save the genuine attempt before protected review.

12. Now you do a small version alone

Fresh case: customers, subscriptions and invoice_lines. Produce one row per subscription with billed value. One customer has multiple subscriptions and one invoice references a missing subscription.

13. Validate your result

State input/output grain, show distinct-key parity, list unmatched keys, and reconcile billed value independently.

14. Teach it back in your own words

Explain why a join can be syntactically correct and analytically wrong.

15. Protected review + changed fresh retry

Save the query and validation first. Then review; close it and retry on a different ticket-event/customer case.

16. Evidence / mastery + exact next route

Runnable SQL + grain/cardinality note + unmatched check + row/total reconciliation + explain-back. VIDEO SUPPORT: Alex The Analyst - 51:10-1:08:03 joins <https://www.postgresql.org/docs/current/tutorial-join.html> After saving evidence, continue to ASQL2.

ASQL2 - Subqueries and derived tables

1. Why it matters

Nested SQL becomes confusing when the inner query has no clear purpose or grain. A useful subquery should simplify one logical stage and make the final question easier to validate.

2. What you are learning

Use scalar, IN/EXISTS and derived-table subqueries appropriately; identify inner result shape; alias derived tables; avoid accidental correlated work when a simpler staged query will do.

3. Prerequisite / what you already know

ASQL1 joins and GROUP BY/aggregates.

4. Picture it in your head

Question inside a question: A subquery is a smaller answer box used by a larger question. First prove the inner result has the grain and columns the outer query expects; then decide whether it belongs in WHERE, FROM, or a scalar expression.

5. New words before we use them

Scalar subquery: A subquery expected to return one value. Derived table: A subquery in FROM treated like a temporary result for that SELECT. Correlated

subquery: A subquery that refers to values from the outer query and may be evaluated in relation to each outer row.

6. Watch / visual source

CURRENT REFERENCE: PostgreSQL 18 - SELECT. Subselect/SELECT behavior.

<https://www.postgresql.org/docs/current/sql-select.html>

Return to the lesson and reproduce the SQL/control from your own practice database. Save course-task evidence, not a screenshot of the source.

7. Follow with me once

- Stage 1: create one row per customer with SUM(order_value) after the approved status filter.
- Inspect that stage alone. Its grain is one row per customer.
- Stage 2: compute AVG(revenue) from that customer-level result. This must return one scalar value.
- Outer query filters customer rows where revenue > the scalar average.
- Validate by saving the scalar average and checking the minimum returned revenue is greater than it.

8. What you should see / be able to say

A nested query where every inner result has a stated shape and can be run independently for debugging.

9. Why it worked

The average is computed at customer grain instead of order grain, so the business question and denominator align.

Professional upgrade: Choose subqueries for compact local logic; choose CTEs when named stages improve review and validation. Maintainability matters more than showing nesting skill.

10. Common beginner mistakes

1. Computing average order value when the question says average customer revenue. | 2. Writing a scalar subquery that returns multiple rows. |
3. Nesting before testing the inner result. | 4. Using subqueries when named CTE stages would be clearer.

11. If you get stuck - recovery ladder

Run the deepest subquery alone. Write its expected columns, row count and grain. Fix it before adding the next level. Save the genuine attempt before protected review.

12. Now you do a small version alone

Fresh case: return products whose monthly revenue is above the average monthly product revenue for the same period.

13. Validate your result

Record the inner average, count returned rows and prove every returned value passes the threshold.

14. Teach it back in your own words

Explain a derived table as a temporary result inside one query without calling it a permanent table.

15. Protected review + changed fresh retry

Attempt first. Changed retry uses EXISTS to find customers with at least one qualifying event.

16. Evidence / mastery + exact next route

Nested SQL + inner-result screenshots/text + independent threshold check + explain-back. VIDEO: intentionally book-led for this lesson

<https://www.postgresql.org/docs/current/sql-select.html> After saving evidence, continue to ASQL3.

ASQL3 - CTEs and readable query architecture

1. Why it matters

A 70-line query with repeated filters can be correct today and dangerous tomorrow. Named stages let another analyst test each transformation and prevent business rules from drifting across repeated fragments.

2. What you are learning

Use WITH CTEs to name logical stages, centralize filters, document stage grain and validate intermediate outputs. Understand that CTEs are query expressions, not automatically stored tables.

3. Prerequisite / what you already know

ASQL2 subqueries and ASQL1 grain control.

4. Picture it in your head

Labeled pipeline stages: A CTE chain is a sequence of named worktables: scope -> clean -> aggregate -> compare -> final. Each stage should have one grain and one purpose, so you can validate it before moving on.

5. New words before we use them

CTE: Common Table Expression: a named query result defined in a WITH clause for use in the statement. Stage grain: The row meaning produced by one CTE stage. Materialization: Whether PostgreSQL stores/evaluates a CTE result separately versus integrating it into planning; behavior can matter for performance.

6. Watch / visual source

RECOMMENDED VIDEO: Alex The Analyst - CTE chapter. 1:58:55-2:09:06.

<https://www.youtube.com/watch?v=OT1RErkfLNQ>

CURRENT REFERENCE: PostgreSQL 18 - WITH Queries. CTE behavior.

<https://www.postgresql.org/docs/current/queries-with.html>

Return to the lesson and reproduce the SQL/control from your own practice database. Save course-task evidence, not a screenshot of the source.

7. Follow with me once

- base: filter source rows once using the approved business rule.
- customer_month: aggregate base to one row per customer + month.
- validated: add only checks or classifications needed before final output; do not hide bad rows.
- final SELECT: choose the business-facing columns and deterministic ORDER BY.
- Run each CTE by temporarily selecting from it to verify row count, distinct keys and control totals.

8. What you should see / be able to say

A query whose stages have meaningful names and whose stage grain can be explained independently.

9. Why it worked

Repeated logic became one controlled stage, making drift and debugging easier to detect. Professional upgrade: In production SQL, stage names often communicate contracts: source_scope, deduped, enriched, customer_day, validated, final. Keep one responsibility per stage where practical.

10. Common beginner mistakes

1. Using CTE names like a,b,c that reveal nothing. | 2. Repeating slightly different status filters in multiple CTEs. | 3. Assuming every CTE improves performance. | 4. Leaving validation only in comments without runnable queries.

11. If you get stuck - recovery ladder

Collapse the query to base only. Add one CTE at a time. After each addition, run a control SELECT from that CTE. Save the genuine attempt before protected review.

12. Now you do a small version alone

Fresh case: build eligible_orders -> customer_week -> ranked_customer_week -> final_exception_report.

13. Validate your result

For every stage save rows, distinct business keys and one control total appropriate to the grain.

14. Teach it back in your own words

Explain why CTEs improve reviewability even when they do not make the query faster.

15. Protected review + changed fresh retry

Attempt first. Changed retry restructures a nested query into named stages.

16. Evidence / mastery + exact next route

CTE query + stage-grain table + stage controls + explain-back. VIDEO SUPPORT: Alex The Analyst - 1:58:55-2:09:06 CTEs <https://www.postgresql.org/docs/current/queries-with.html> After saving evidence, continue to ASQL4.

ASQL4 - Window functions fundamentals

1. Why it matters

GROUP BY collapses rows. Many business questions need a group calculation while still keeping each row - for example each order plus the customer's total or department average.

2. What you are learning

Recognize when a window function is needed; use OVER, PARTITION BY and ORDER BY; explain why row identity is retained; filter window results through a later query stage when necessary.

3. Prerequisite / what you already know

Aggregates and ASQL3 staged queries.

4. Picture it in your head

Keep the rows, add a lens: GROUP BY collapses rows; a window function keeps the underlying rows and adds a calculated lens across related rows. State the partition and order before choosing the window expression.

5. New words before we use them

Window function: A calculation across related rows that keeps the individual output rows. PARTITION BY: Divides result rows into groups for the window calculation. OVER: Marks the function as a window calculation and defines its window.

6. Watch / visual source

RECOMMENDED VIDEO: Alex The Analyst - Window Functions chapter. 1:45:58-1:58:54.

<https://www.youtube.com/watch?v=OT1RErkfLNQ>

CURRENT REFERENCE: PostgreSQL 18 - Window Functions tutorial. Window mental model.

<https://www.postgresql.org/docs/current/tutorial-window.html>

Return to the lesson and reproduce the SQL/control from your own practice database. Save course-task evidence, not a screenshot of the source.

7. Follow with me once

- Start with eligible orders at one row per order.
- Add SUM(order_value) OVER (PARTITION BY customer_id) AS customer_revenue.
- Compare output row count to input row count; it should remain one row per eligible order.
- Pick one customer and independently GROUP BY customer_id to confirm the repeated customer_revenue value.
- If you need top rows based on a window result, calculate the window in a subquery/CTE and filter in the outer SELECT.

8. What you should see / be able to say

Window result added without collapsing order rows, with a saved independent grouped check.

9. Why it worked

OVER changes the aggregate from row-collapsing behavior to a calculation over a defined set while rows retain identity.

Professional upgrade: Window

functions are excellent for analytical comparisons, but partition, order and frame are part of the business definition and should be reviewed like any other metric logic.

10. Common beginner mistakes

1. Expecting window SUM to reduce rows. | 2. Putting window functions directly in WHERE. | 3. Using PARTITION BY when the business question needs the whole result set. | 4. Forgetting that window ORDER BY affects calculation order, not necessarily final display order.

11. If you get stuck - recovery ladder

Complete the matching course task and preserve your evidence before moving on.

12. Now you do a small version alone

Fresh case: show every ticket with team average resolution time while keeping one row per ticket.

13. Validate your result

Base/output rows equal; one team's window value matches independent GROUP BY result.

14. Teach it back in your own words

Explain GROUP BY versus a window function using 'collapse rows' versus 'keep rows'.

15. Protected review + changed fresh retry

Attempt first. Changed retry uses each transaction plus account-level total.

16. Evidence / mastery + exact next route

Runnable window SQL + base/output row parity + independent partition check + explain-back. VIDEO SUPPORT: Alex The Analyst - 1:45:58-1:58:54

window functions <https://www.postgresql.org/docs/current/tutorial-window.html> After saving evidence, continue to ASQL5.

ASQL5 - Ranking, partitions and running calculations

1. Why it matters

Top-N and running totals look simple until ties or duplicate timestamps appear. ROW_NUMBER, RANK and DENSE_RANK answer different questions, and default window frames can surprise you.

2. What you are learning

Choose ranking function based on tie policy; define deterministic ordering; distinguish partition from frame; write explicit ROWS frames for running calculations when needed.

3. Prerequisite / what you already know

ASQL4 window basics.

4. Picture it in your head

Resettable race track: PARTITION BY creates separate tracks; ORDER BY decides the race order inside each track; ROW_NUMBER/RANK/DENSE_RANK decide how positions behave when values tie. A running calculation also needs an explicit frame.

5. New words before we use them

ROW_NUMBER: Unique sequential number according to the specified window order. RANK: Ranking with gaps after ties. DENSE_RANK: Ranking without gaps after ties. Frame: The subset of rows within the window used for a particular calculation.

6. Watch / visual source

CURRENT REFERENCE: PostgreSQL 18 - Window Functions. Ranking/frame functions.
<https://www.postgresql.org/docs/current/functions-window.html>

Return to the lesson and reproduce the SQL/control from your own practice database. Save course-task evidence, not a screenshot of the source.

7. Follow with me once

- For top order value per customer, decide tie policy before picking ROW_NUMBER/RANK/DENSE_RANK.
- Add a deterministic tie-breaker such as order_id after order_value if exactly one row must win.
- For running spend, order by order_date, order_id and write ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW explicitly.
- Select one customer and manually sort their orders to verify the first three running totals.
- Document whether two equal order values should share rank or one must be chosen deterministically.

8. What you should see / be able to say

Ranking and running-total logic matches a written tie/frame policy.

9. Why it worked

The business rule controls ranking and window frame instead of relying on defaults. Professional upgrade: Stable analytical outputs use deterministic sort keys. If the decision requires exactly N rows, document how ties are broken.

10. Common beginner mistakes

1. Using ROW_NUMBER when ties should share a rank. | 2. Ordering by a non-unique timestamp without tie-breaker. | 3. Assuming default frame always means 'rows so far'. | 4. Using final ORDER BY as if it changed window calculation order.

11. If you get stuck - recovery ladder

Create a tiny 5-row example with an intentional tie. Compare ROW_NUMBER/RANK/DENSE_RANK. Then test the frame on the same rows. Save the genuine attempt before protected review.

12. Now you do a small version alone

Fresh case: top two products per region by revenue plus cumulative monthly revenue within region.

13. Validate your result

Check tie case explicitly and manually recompute one partition's running values.

14. Teach it back in your own words

Explain why a ranking function choice is a business rule, not only syntax.

15. Protected review + changed fresh retry

Attempt first. Changed retry uses employee performance scores with deliberate ties.

16. Evidence / mastery + exact next route

Ranking SQL + tie policy + explicit frame + manual partition check + explain-back. VIDEO: intentionally book-led for this lesson

<https://www.postgresql.org/docs/current/functions-window.html> After saving evidence, continue to ASQL6.

ASQL6 - LAG, LEAD and time-based comparisons

1. Why it matters

Month-over-month or event-to-event comparison is wrong if the rows are not first reduced to the correct time grain and ordered deterministically.

2. What you are learning

Use LAG/LEAD after defining entity and time grain; handle the first/last row; distinguish missing period from zero; calculate changes only after validating ordering and continuity.

3. Prerequisite / what you already know

ASQL5 window ordering and ASQL8-style typed dates will reinforce this lesson.

4. Picture it in your head

Time-neighbor ruler: LAG and LEAD let each row look to a defined neighbor in an ordered series. The ruler is meaningful only when the time grain is complete and the ordering key is deterministic; missing periods do not magically appear.

5. New words before we use them

LAG: Returns a value from a previous row in the window ordering. LEAD: Returns a value from a following row in the window ordering. Time grain: The time unit represented by one row, such as one row per month.

6. Watch / visual source

OPTIONAL VIDEO: Alex The Analyst - Window Functions chapter. 1:45:58-1:58:54.

<https://www.youtube.com/watch?v=OT1RErkfLNQ>

CURRENT REFERENCE: PostgreSQL 18 - Window Functions. LAG/LEAD.

<https://www.postgresql.org/docs/current/functions-window.html>

Return to the lesson and reproduce the SQL/control from your own practice database. Save course-task evidence, not a screenshot of the source.

7. Follow with me once

- First aggregate eligible orders to one row per month using `date_trunc('month', order_date)`.
- Apply `LAG(revenue) OVER (ORDER BY month_start)` to the monthly result, not raw order rows.
- Compute absolute change and, only when denominator policy is defined, percentage change.
- Check the first month: `prior_revenue` should be NULL because no earlier row exists.
- Detect whether a missing calendar month is absent from the data. Missing row is not the same as a zero-revenue month.

8. What you should see / be able to say

One row per month with prior value/change and an explicit policy for first/missing periods.

9. Why it worked

The time series is normalized to month grain before row-to-row comparison. Professional upgrade: For operational time series, a calendar/date spine may be needed to represent missing periods explicitly. That deeper modeling appears later; here you must at least detect the difference.

10. Common beginner mistakes

1. Using LAG on unsummarized order rows for monthly change. | 2. Treating NULL prior value as zero automatically. | 3. Ignoring missing months. | 4. Ordering by formatted month text instead of typed time.

11. If you get stuck - recovery ladder

Run the monthly CTE alone. Confirm one row per period and chronological order. Only then add LAG/LEAD. Save the genuine attempt before

protected review.

12. Now you do a small version alone

Fresh case: weekly support volume by team with previous-week comparison; one week is missing for one team.

13. Validate your result

Manually check two adjacent periods and explicitly flag the missing-period condition.

14. Teach it back in your own words

Explain why LAG does not know what 'previous month' means unless your row set and order make it true.

15. Protected review + changed fresh retry

Attempt first. Changed retry compares next scheduled appointment per patient with LEAD.

16. Evidence / mastery + exact next route

Time-grain query + LAG/LEAD + missing-period check + manual comparison + explain-back. VIDEO SUPPORT: Alex The Analyst - 1:45:58-1:58:54

optional window visual; LAG/LEAD taught in-book <https://www.postgresql.org/docs/current/functions-window.html> After saving evidence, continue to

ASQL7.

ASQL7 - Conditional aggregation and advanced KPI queries

1. Why it matters

A KPI query can return a neat percentage while numerator and denominator use different populations. Conditional aggregation must implement the metric contract visibly.

2. What you are learning

Use CASE and PostgreSQL FILTER for conditional counts/sums; separate numerator and denominator; use NULLIF for defined zero-denominator behavior; reconcile KPI components independently.

3. Prerequisite / what you already know

C2-00 metric contracts, aggregates, ASQL1 grain.

4. Picture it in your head

One scan, many buckets: Conditional aggregation sends each input row into named KPI buckets using CASE/FILTER rules, then aggregates once. Write each numerator and denominator rule in words before encoding it.

5. New words before we use them

Conditional aggregation: Aggregating only rows that satisfy a condition, often through CASE or FILTER. FILTER clause: PostgreSQL syntax that applies a WHERE-like condition to one aggregate. NULLIF: Returns NULL when two expressions are equal; often used to avoid division by zero after the business rule is defined.

6. Watch / visual source

RECOMMENDED VIDEO: Alex The Analyst - CASE Statements chapter. 1:26:39-1:35:15.

<https://www.youtube.com/watch?v=OT1RErkfLNQ>

CURRENT REFERENCE: PostgreSQL 18 - Aggregate Functions. Aggregate behavior.

<https://www.postgresql.org/docs/current/functions-aggregate.html>

Return to the lesson and reproduce the SQL/control from your own practice database. Save course-task evidence, not a screenshot of the source.

7. Follow with me once

- Write the metric contract first: numerator = completed eligible orders; denominator = all eligible non-test orders.
- Use COUNT(*) FILTER (WHERE status='completed') for numerator when PostgreSQL is the target dialect.
- Build denominator separately with its own eligible-population filter; do not assume COUNT(*) is correct.
- Compute rate using numerator::numeric / NULLIF(denominator,0) only after zero-population behavior is agreed.
- Reconcile numerator and denominator for one region with independent WHERE queries.

8. What you should see / be able to say

KPI output whose components can be inspected and independently recomputed.

9. Why it worked

Business definitions stay visible in the aggregate expressions instead of being hidden in a final ratio. Professional upgrade: In reviewed SQL, metric components are often named in a CTE so definitions can be tested and reused. Keep business labels close to the exact filters they mean.

10. Common beginner mistakes

1. Different filters on numerator and denominator by accident. | 2. ELSE NULL versus ELSE 0 confusion without understanding aggregate behavior. | 3. IFERROR-like hiding of zero denominator. | 4. Formatting as percent before validating components.

11. If you get stuck - recovery ladder

Complete the matching course task and preserve your evidence before moving on.

12. Now you do a small version alone

Fresh case: First Contact Resolution Rate by team with an exclusion for reopened-within-48-hours cases.

13. Validate your result

Independent numerator/denominator queries match output; sum of segments reconciles to overall eligible population where appropriate.

14. Teach it back in your own words

Explain why a percentage should be built last, not first.

15. Protected review + changed fresh retry

Attempt first. Changed retry builds refund rate with a different denominator.

16. Evidence / mastery + exact next route

KPI SQL + metric contract + component validation + explain-back. VIDEO SUPPORT: Alex The Analyst - 1:26:39-1:35:15
CASE statements

<https://www.postgresql.org/docs/current/functions-aggregate.html> After saving evidence, continue to ASQL8.

ASQL8 - Date and time analysis

1. Why it matters

Text slicing and inclusive end dates often create hidden period errors. Timestamps also introduce time-zone and boundary questions that must be explicit.

2. What you are learning

Use typed DATE/TIMESTAMP values, half-open intervals, date_trunc/date_bin where appropriate, and explicit time-zone assumptions. Avoid text-based month logic.

3. Prerequisite / what you already know

Basic WHERE filters; ASQL6 time-grain reasoning.

4. Picture it in your head

Half-open calendar boxes: Put timestamps into boxes such as [start, next_start). This includes the first instant and excludes the next period boundary, avoiding 23:59:59 tricks and making month/week logic safe across timestamp precision.

5. New words before we use them

Half-open interval: A time filter including the start and excluding the next-period boundary, such as $\geq$ Apr 1 and $<$ May 1.
date_trunc: PostgreSQL function that truncates a date/time value to a requested precision. Time zone: Rules used to interpret local clock time relative to UTC/other zones.

6. Watch / visual source

CURRENT REFERENCE: PostgreSQL 18 - Date/Time Functions and Operators. date_trunc/date-time behavior.
<https://www.postgresql.org/docs/18/functions-datetime.html>

Return to the lesson and reproduce the SQL/control from your own practice database. Save course-task evidence, not a screenshot of the source.

7. Follow with me once

- Use order_ts $\geq$ TIMESTAMP '2026-01-01 00:00:00' AND order_ts $<$ TIMESTAMP '2026-07-01 00:00:00'.
- Bucket with date_trunc('month', order_ts) after confirming the business reporting time zone.
- Do not use BETWEEN '2026-01-01' AND '2026-06-30' for timestamp data unless you truly intend the end literal behavior.
- Check minimum and maximum timestamps returned and count rows exactly at a boundary in a tiny test.
- If source is timestampz, record the reporting-zone decision before grouping by local month.

8. What you should see / be able to say

Typed, boundary-safe period logic with explicit calendar/time-zone assumptions.

9. Why it worked

The next-period exclusive boundary includes every instant in the intended period without guessing the final day's maximum time. Professional upgrade:

Period logic belongs in metric contracts. Reporting month, operational day and UTC storage can all be different concepts.

10. Common beginner mistakes

1. Comparing dates stored as text. | 2. Using $\leq$ end_date with timestamp data carelessly. | 3. Grouping UTC timestamps into local business days without a zone decision. | 4. Formatting first, then grouping on strings.

11. If you get stuck - recovery ladder

Inspect column data type and sample minimum/maximum. Test one record at the start boundary and one at the next-period boundary. Save the genuine attempt before protected review.

12. Now you do a small version alone

Fresh case: analyze support events by local business day when timestamps are stored with time zone.

13. Validate your result

Boundary test rows behave as predicted; bucket count reconciles to filtered source count.

14. Teach it back in your own words

Explain half-open date filtering using a shop open from one boundary until the next.

15. Protected review + changed fresh retry

Attempt first. Changed retry uses weekly buckets with a different reporting zone.

16. Evidence / mastery + exact next route

Typed date/time SQL + boundary tests + bucket reconciliation + explain-back. VIDEO: intentionally book-led for this lesson <https://www.postgresql.org/docs/18/functions-datetime.html> After saving evidence, continue to ASQL9.

ASQL9 - Data-quality investigation with SQL

1. Why it matters

Data quality should be investigated before it contaminates a metric. SQL can turn vague suspicion into exact exception sets that can be counted, owned and repaired.

2. What you are learning

Write reusable checks for duplicate business keys, null required fields, orphan keys, invalid ranges, unexpected categories and cross-table control totals.

3. Prerequisite / what you already know

ASQL1 joins/grain and basic aggregates.

4. Picture it in your head

Exception flashlight: Data-quality SQL is not trying to hide bad rows; it shines a light on duplicates, nulls, orphans, impossible values and reconciliation breaks. Keep an exception result beside the cleaned result.

5. New words before we use them

Profiling query: A query that summarizes shape, values or exceptions to understand data quality. Required field: A field whose missing value makes a business process or analysis invalid. Domain rule: A rule defining valid values/ranges for a field.

6. Watch / visual source

CURRENT REFERENCE: PostgreSQL 18 - Table and Query Expressions. Filtering/join expressions.
<https://www.postgresql.org/docs/current/queries-table-expressions.html>

Return to the lesson and reproduce the SQL/control from your own practice database. Save course-task evidence, not a screenshot of the source.

7. Follow with me once

- Duplicate customer key: GROUP BY customer_id HAVING COUNT(*) > 1.
- Missing required field: WHERE region IS NULL OR btrim(region)="".
- Orphan order: LEFT JOIN customers and filter c.customer_id IS NULL.
- Invalid amount: WHERE order_value < 0 when negatives are not allowed by the contract.
- Unexpected status: compare DISTINCT status against an approved status list; save exception counts rather than silently mapping them.

8. What you should see / be able to say

A small exception report where each quality rule has count, sample keys and business impact.

9. Why it worked

The checks target different failure classes instead of relying on one generic 'clean data' query. Professional upgrade: Quality checks become operational when each rule has owner, severity, expected threshold and recovery action. C2-02 focuses on query design; broader observability comes later.

10. Common beginner mistakes

1. Removing duplicate rows before identifying which key is duplicated. | 2. Treating all nulls as errors. | 3. Using NOT IN carelessly when null semantics are not understood. | 4. Fixing source values directly without owner approval.

11. If you get stuck - recovery ladder

Choose one rule at a time: grain/key, required field, relationship, range, category. Return only exception rows and count them. Save the genuine attempt before protected review.

12. Now you do a small version alone

Fresh case: investigate ticket, team and customer tables containing duplicate teams, orphan customer IDs and negative resolution hours.

13. Validate your result

Exception counts are reproducible and zero only after an approved repair; raw evidence remains preserved.

14. Teach it back in your own words

Explain why data-quality SQL should return the bad rows, not only a green/red flag.

15. Protected review + changed fresh retry

Attempt first. Changed retry uses subscription/billing quality rules.

16. Evidence / mastery + exact next route

Quality-check SQL + exception outputs + business-impact note + explain-back. VIDEO: intentionally book-led for this lesson <https://www.postgresql.org/docs/current/queries-table-expressions.html> After saving evidence, continue to ASQL10.

ASQL10 - Query debugging and validation

1. Why it matters

Changing several clauses at once can make a query appear fixed without proving which error caused the problem. Senior debugging isolates the first broken stage.

2. What you are learning

Classify syntax versus logic versus data-quality failures; debug stage-by-stage; compare row count/distinct keys/control totals; build validation queries that could catch plausible wrong answers.

3. Prerequisite / what you already know

ASQL1-ASQL9; especially CTE stages and grain checks.

4. Picture it in your head

Query autopsy checkpoints: When a final total is wrong, do not stare at the last SELECT. Check source population -> filters -> joins -> grain -> aggregation -> window -> final formatting, and stop at the first stage where a control changes unexpectedly.

5. New words before we use them

Syntax error: The database cannot parse/execute the statement as written. Logic error: The SQL runs but answers the wrong question. Control total: An independently obtained count/sum used to reconcile the query result.

6. Watch / visual source

CURRENT REFERENCE: PostgreSQL 18 - SELECT. Query staging/debug context.

<https://www.postgresql.org/docs/current/sql-select.html>

Return to the lesson and reproduce the SQL/control from your own practice database. Save course-task evidence, not a screenshot of the source.

7. Follow with me once

- Record the symptom: final revenue is almost exactly 2x expected.
- Run base CTE controls: rows, distinct order_id, SUM(order_value).
- Run the next join stage with the same controls. If rows and total double here, stop; later stages are not the first cause.
- Inspect right-side join key for duplicates. Do not change filters or add DISTINCT yet.
- Repair the key/cardinality cause, rerun controls from base through final, and save before/after evidence.

8. What you should see / be able to say

A debugging log identifying the first failed stage and proving the repair removed the underlying cause.

9. Why it worked

The search space shrinks because every stage has shape and total evidence. Professional upgrade: Keep small diagnostic queries near complex analytical SQL during development. Validation queries are part of the deliverable for important outputs.

10. Common beginner mistakes

1. Editing many clauses before rerunning controls. | 2. Fixing the final SELECT when the error began upstream. | 3. Using the same final query as its own validation. | 4. Deleting problem rows until totals match.

11. If you get stuck - recovery ladder

Return to the earliest stable stage. Add one transformation at a time and compare rows, distinct keys and one additive total. Save the genuine attempt before protected review.

12. Now you do a small version alone

Fresh case: a monthly customer report loses 12 customers after enrichment. Find the first stage where row count drops and decide whether the loss is expected.

13. Validate your result

Before/after evidence shows exactly which stage changed shape and why; final controls rerun after repair.

14. Teach it back in your own words

Explain why 'the query runs now' is not proof that the bug is fixed.

15. Protected review + changed fresh retry

Attempt first. Changed retry uses an incorrect date filter rather than a join bug.

16. Evidence / mastery + exact next route

Debug log + stage controls + root cause + repair + rerun evidence + explain-back. VIDEO: intentionally book-led for this lesson

<https://www.postgresql.org/docs/current/sql-select.html> After saving evidence, continue to ASQL11.

ASQL11 - Performance concepts: indexes, scans and EXPLAIN

1. Why it matters

Optimization before correctness can make the wrong result faster. Analysts need enough plan awareness to identify obvious scans/row-estimate problems and communicate evidence without pretending to be database administrators.

2. What you are learning

Read basic EXPLAIN nodes/cost/estimated rows; understand sequential versus index scans conceptually; know that EXPLAIN ANALYZE executes the statement; compare estimated and actual rows; discuss index trade-offs.

3. Prerequisite / what you already know

Correct, validated SQL from earlier lessons. Basic idea of a database index is taught here.

4. Picture it in your head

Route map before stopwatch: EXPLAIN is the planner's route map; EXPLAIN ANALYZE adds actual runtime evidence. First ask whether the query is logically correct and selective, then inspect scans/joins/index use. A faster wrong query is still wrong.

5. New words before we use them

Query plan: The execution strategy PostgreSQL chooses for a statement. Sequential scan: Reading table pages broadly rather than locating rows via an index. Index scan: Using an index to locate candidate rows. Estimate: Planner prediction such as expected rows/cost before actual execution.

6. Watch / visual source

CURRENT REFERENCE: PostgreSQL 18 - Using EXPLAIN. EXPLAIN / EXPLAIN ANALYZE.

<https://www.postgresql.org/docs/current/using-explain.html>

CURRENT REFERENCE: PostgreSQL 18 - Indexes. Index concepts/trade-offs.

<https://www.postgresql.org/docs/current/indexes.html>

Return to the lesson and reproduce the SQL/control from your own practice database. Save course-task evidence, not a screenshot of the source.

7. Follow with me once

- First prove the SELECT is correct with normal validation.
- Run EXPLAIN SELECT ... to inspect the planned access path without requesting actual execution details.
- Run EXPLAIN (ANALYZE, BUFFERS) only on a safe SELECT in the practice environment and remember ANALYZE executes it.
- Compare estimated rows to actual rows and identify the largest mismatch or expensive-looking node; do not optimize from node names alone.
- Discuss whether an index on the filter/join key could help and what write/storage overhead it adds. Measure before and after if the lab provides
- an index.

8. What you should see / be able to say

A short plan note naming observed scan/join type, estimate-versus-actual evidence and one measured optimization hypothesis.

9. Why it worked

Performance reasoning comes after correctness and uses observed plan evidence rather than generic 'indexes make SQL fast' claims. Professional upgrade:

Analysts should know when to escalate a performance issue with reproducible SQL, row counts and EXPLAIN evidence rather than guessing at database configuration.

10. Common beginner mistakes

1. Running EXPLAIN ANALYZE on unsafe modifying statements without understanding execution. | 2. Calling every sequential scan bad. | 3. Creating many indexes without workload evidence. | 4. Comparing timing from unrelated environments as if absolute.

11. If you get stuck - recovery ladder

Use plain EXPLAIN first. Identify top nodes, estimated rows and predicates. If safe, add ANALYZE in the lab. Compare one change at a time. Save the genuine attempt before protected review.

12. Now you do a small version alone

Fresh case: compare a selective date/customer filter before and after an approved practice index. Record plan and timing/row evidence.

13. Validate your result

Query result stays identical; plan evidence saved; trade-off statement includes index maintenance/storage cost.

14. Teach it back in your own words

Explain why a sequential scan can sometimes be the right plan.

15. Protected review + changed fresh retry

Attempt first. Changed retry examines a join query with a different selectivity pattern.

16. Evidence / mastery + exact next route

Validated query + before/after EXPLAIN evidence + result parity + trade-off + explain-back. VIDEO: intentionally book-led for this lesson

<https://www.postgresql.org/docs/current/using-explain.html> After saving evidence, continue to ASQL12.

ASQL12 - Maintainable production-style analytical SQL

1. Why it matters

A one-off query may answer today. A production-style analytical query must let another analyst understand the decision, assumptions, grain, stages, validation and how to rerun it safely.

2. What you are learning

Package analytical SQL with header contract, readable CTE stages, consistent names/formatting, deterministic output, separate validations, business interpretation and handoff notes.

3. Prerequisite / what you already know

All ASQL1-ASQL11.

4. Picture it in your head

Production SQL assembly line: Inputs and assumptions enter, named stages transform them, controls verify row/money contracts, and a documented final output leaves the line. Maintainability means another analyst can rerun and audit it without guessing.

5. New words before we use them

Deterministic output: Output whose ordering/tie behavior is explicitly controlled when order matters. Query contract: Short documentation of decision, scope, grain, assumptions and source tables. Handoff: Information another analyst needs to rerun, validate and modify the analysis safely.

6. Watch / visual source

OPTIONAL VIDEO: Alex The Analyst - Data Cleaning / EDA project chapters. 2:42:46-3:32:13.

<https://www.youtube.com/watch?v=OT1RErkfLNQ>

CURRENT REFERENCE: PostgreSQL 18 - Queries. Query architecture.

<https://www.postgresql.org/docs/current/queries.html>

Return to the lesson and reproduce the SQL/control from your own practice database. Save course-task evidence, not a screenshot of the source.

7. Follow with me once

- Header comment: decision, metric definition/version, intended final grain, source tables, period, exclusions and assumptions.
- WITH stages: base_scope -> quality_checked -> enriched -> business_grain -> final_metrics. Use descriptive aliases.
- Final SELECT includes only decision-facing columns and deterministic ORDER BY when presentation order matters.
- Separate validation block checks row/distinct-key counts, orphans and control totals; do not hide it inside comments.
- Write a 3-5 sentence interpretation: what happened, materiality, uncertainty, recommendation and what would change it.

8. What you should see / be able to say

A runnable SQL package another analyst can review stage-by-stage and reconcile without asking what each section means.

9. Why it worked

Technical logic, business contract and evidence are packaged together instead of living in separate memory/chat fragments. Professional upgrade:

Maintainable SQL is boring in a good way: explicit contracts, stable naming, small stages, deterministic logic, validation and evidence. Complexity is justified only when the business problem requires it.

10. Common beginner mistakes

1. Leaving magic dates/status values unexplained. | 2. Using SELECT * in the final output without need. | 3. No deterministic tie/order rule. | 4. Deleting validation queries before handoff. | 5. Writing business conclusions not supported by the query.

11. If you get stuck - recovery ladder

Start with the header contract and final grain. Rename each stage for one job. Add one validation per major failure mode. Only then polish formatting. Save the genuine attempt before protected review.

12. Now you do a small version alone

Fresh case: deliver a production-style regional service-performance query with quality exceptions, KPI output and a concise business recommendation.

13. Validate your result

A second analyst can state source/grain/rules, run validations, reproduce final output and identify one safe modification point.

14. Teach it back in your own words

Explain what makes SQL 'production-style' without using line count or advanced syntax as the definition.

15. Protected review + changed fresh retry

Attempt first. Changed retry packages a different subscription-health analysis.

16. Evidence / mastery + exact next route

Final SQL file + validation section + data-quality note + interpretation + handoff + 90-second defense. VIDEO SUPPORT: Alex The Analyst -

2:42:46-3:32:13 optional cleaning/project workflow <https://www.postgresql.org/docs/current/queries.html> After saving evidence, complete the module

mastery checklist and follow the Gate A / completion route.